

Universidade Federal de Pernambuco

Centro de Informática - CIn

Acidentes de trânsito com vítima em Recife (2014-2016)



Relatório do Projeto de Integração Banco de dados - 2026.1

Anne Pereira Collier de Mendonça <apcm>
Arthur Sean Cerqueira Campos <ascc2>
Arthur Fidney Da Cunha Cavalcante Correia <afccc>
Isabela Possídio Amorim <ipa>
João Guilherme Lemos Duarte <jgldo>
João Luís Gomes Agra <jlga>

1. Descrição da Base de Dados Unificada

A base de dados unificada foi modelada utilizando a abordagem dimensional, adotando um esquema estrela para organizar e disponibilizar os dados de acidentes de trânsito com vítimas ocorridos entre os anos de 2014 e 2016. O objetivo desse modelo é facilitar consultas analíticas, agregações e geração de indicadores relacionados aos acidentes registrados.

O esquema estrela é composto por uma tabela fato central, denominada **fato_acidente**, e cinco tabelas dimensão responsáveis por armazenar os atributos descritivos associados aos eventos registrados.

- Tabela Fato: fato_acidente

A tabela fato representa cada ocorrência de acidente registrada na base integrada. Ela centraliza as medidas quantitativas e referencia as dimensões por meio de chaves substitutas.

Atributos:

Atributo	Tipo	Descrição
sk_acidente	INT	Chave primária da tabela fato
sk_tempo	INT	Chave estrangeira para a dimensão tempo
sk_local	INT	Chave estrangeira para a dimensão local
sk_tipo_acidente	INT	Chave estrangeira para a dimensão tipo de acidente
sk_ocorrendia	INT	Chave estrangeira para a dimensão ocorrência
sk_veiculo	INT	Chave estrangeira para a dimensão veículo
quantidade_vitimas	INT	Quantidade de vítimas registradas no acidente
quantidade_veiculos	INT	Quantidade de veículos envolvidos na ocorrência

As métricas principais da tabela fato são a quantidade de vítimas e a quantidade de veículos envolvidos, permitindo análises estatísticas e comparativas.

- **Dimensão Tempo (dim_tempo)**

A dimensão tempo foi criada para possibilitar análises temporais dos acidentes.

Atributos:

- sk_tempo
- data_acidente
- ano
- mês
- dia
- hora_acidente
- dia_semana
- fim_de_semana

Essa dimensão permite análises por período, sazonalidade, dias da semana e horários de maior incidência de acidentes.

- **Dimensão Local (dim_local)**

A dimensão local armazena as informações geográficas relacionadas ao local da ocorrência.

Atributos:

- sk_local
- bairro
- endereco
- latitude
- longitude

Essa dimensão possibilita a realização de análises espaciais, identificação de áreas críticas e visualização geográfica dos acidentes.

- **Dimensão Tipo de Acidente (dim_tipo_acidente)**

Responsável por armazenar informações sobre a natureza e características do acidente.

Atributos:

- sk_tipo_acidente
- causa_acidente
- descricao_detalhada

A utilização desta dimensão permite analisar as principais causas e características dos acidentes registrados.

- **Dimensão Ocorrência (dim_ocorrencia)**

Armazena informações relacionadas à classificação administrativa da ocorrência.

Atributos:

- sk_ocorrencia
- tipo_ocorrencia
- classificacao_ocorrencia

Essa dimensão possibilita segmentar os acidentes conforme sua categoria operacional e classificação institucional.

- Dimensão Veículo (dim_veiculo)

Contém informações referentes aos veículos envolvidos nas ocorrências.

Atributos:

- sk_veiculo
- tipo_veiculo
- categoria_veiculo

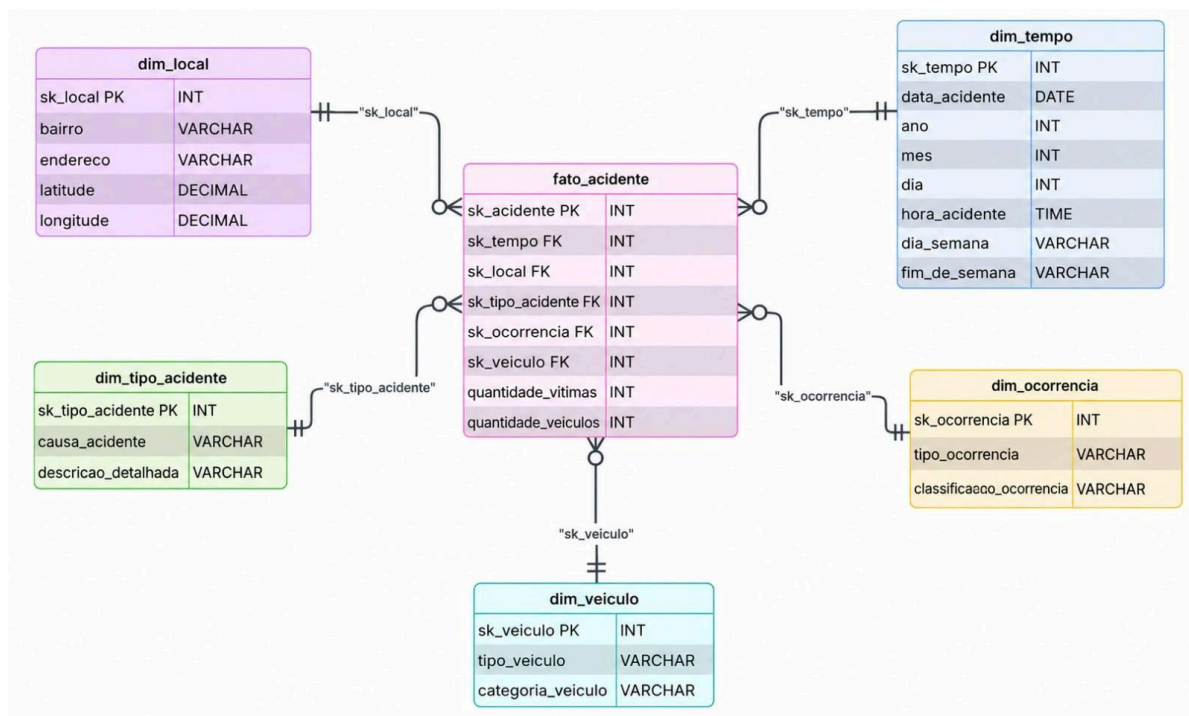
A dimensão veículo permite análises sobre os tipos de veículos mais frequentemente envolvidos em acidentes e suas respectivas categorias.

Metadados e Padronização

Durante o processo de integração foi necessária a padronização dos dados provenientes de diferentes formatos (CSV e GeoJSON) e diferentes versões da base original. Campos equivalentes presentes em anos distintos foram mapeados para atributos únicos do modelo dimensional.

As datas foram convertidas para um formato padronizado, os campos numéricos foram normalizados e as coordenadas geográficas foram armazenadas em formato decimal. Valores inconsistentes encontrados nos arquivos originais foram tratados durante a etapa de transformação dos dados.

A utilização de chaves substitutas nas dimensões garante independência em relação aos identificadores originais dos arquivos-fonte, proporcionando maior flexibilidade e desempenho nas consultas analíticas realizadas sobre o Data Warehouse.



[Figura 01 - Esquema Estrela]

2. Explicação Detalhada do Processo de Integração:

- ETL (Extração, Transformação e Carga):

Extração:

Na etapa de extração na técnica de ETL, os dados foram importados do Google Drive e lidos usando a biblioteca pandas. Entre eles, foram incluídos arquivos mensais de 2015 e anuais, que apresentavam estruturas diferentes como variações dos separadores (: ou ,) e divergências nos nomes e quantidades de colunas.

Transformação:

Na etapa de transformação, foram criados dicionários (map_2014, map_2015, map_2016) que padronizam as colunas de cada arquivo através de um mapeamento para um formato comum.

Adicionalmente, a função "processar_arquivo_mensal" foi escrita para automatizar a limpeza dos arquivos mensais (2015), padronizar todas as colunas para letras minúsculas, renomear e filtrar apenas as colunas desejadas evitando erros na união.

Além disso, valores vazios em colunas de texto foram preenchidos com "Não Informado" e valores nulos na coluna quantidade_vitimas foram preenchidos com 1. Já a coluna de data foi convertida para o formato datetime do pandas para evitar erros.

As colunas tipo_acidente, veiculo e bairro tiveram seus textos convertidos para letras maiúsculas e espaços extras no fim foram removidos para garantir a integridade dos dados.

A partir da base consolidada, combinações únicas dos dados foram extraídas a fim de criar as tabelas dimensão (Tipo Acidente, Veículo, Local, Tempo). Para cada dimensão, foi gerada uma chave primária sequencial (sk_), utilizando o índice do DataFrame.

A dimensão do tempo foi expandida pela extração do ano, mês e dia, além da condição de ser um dia da semana (segunda a sexta), indicada por uma flag booleana.

Somado a isso, vários merges foram feitos com as tabelas dimensão para capturar as chaves artificiais. Após isso, as colunas descritivas de texto foram removidas, deixando a Fato apenas com as chaves e métricas numéricas.

Carga:

Inicialmente, todos os DataFrames modelados (Dimensões e Fato) foram exportados como arquivos locais .csv para backup e fácil acesso. Para a consolidação final, utilizou-se a biblioteca nativa sqlite3 para construir o banco de dados local (dw_acidentes.db). A arquitetura do banco foi definida rigorosamente por meio de scripts DDL (CREATE TABLE), respeitando a ordem de criação (primeiro as dimensões, seguidas pela tabela fato) e ativando explicitamente a verificação estrutural com o comando PRAGMA foreign_keys = ON; para garantir a integridade relacional.

Antes da inserção, os DataFrames em Python passaram por uma etapa de formatação estrita para se adequarem ao banco. Isso incluiu o renomeamento de atributos para espelhar o modelo físico (como hora para hora_acidente) e a instanciação de colunas faltantes com valores nulos (None) em dimensões que possuíam atributos não preenchidos inicialmente (como descricao_detalhada e latitude/longitude em alguns contextos). Além disso, os DataFrames foram filtrados para conter exclusivamente as colunas mapeadas no SQL.

Por fim, a inserção dos dados nas tabelas estruturadas ocorreu de forma segura utilizando a função .to_sql(if_exists='append', index=False) da biblioteca Pandas. Para validar o encerramento do pipeline, foram executadas queries SQL automáticas de verificação de qualidade, as quais confirmaram o total de registros por tabela e atestaram a integridade referencial perfeita (zero chaves estrangeiras órfãs na Tabela Fato).

- ELT (Extração,Carga e Transformação):

Extração:

Na fase de extração na técnica de ELT, foram mapeadas 8 fontes de dados nativas do período de 2014 a 2016, nas quais os formatos extraídos são heterogêneos, consistindo em arquivos .csv (2014, 2015 anual, abril/2015, maio/2015 e 2016) e .geojson (janeiro, fevereiro e março de 2015).

Carga:

Na etapa de carga, os dados foram armazenados diretamente no DuckDB local dentro de um schema chamado raw, em 8 tabelas distintas: acid_2014, acid_2015_vitimas, acid_2015_abril, acid_2015_maio, acid_2016, acid_2015_jan, acid_2015_fev e acid_2015_mar. Em especial para os arquivos GeoJSON, aplicou-se a função unnest(features) para desaninhar as propriedades e coordenadas.

Adicionalmente, nenhuma transformação de tipo foi feita nesta etapa, em que todas as colunas foram carregadas como texto (all_varchar=true).

Transformação:

Na transformação, aconteceu a união das fontes brutas em uma única tabela contendo a identificação da origem dos dados. Foram utilizadas múltiplas chamadas de try_strptime, em alguns casos combinadas com COALESCE, para acomodar os diferentes formatos de data encontrados nas fontes em um padrão DATE único, filtrando a janela entre 2014 e 2016. Adicionalmente, foram utilizadas expressões regulares e as funções upper e trim para limpar strings, além da conversão de latitudes e longitudes para DOUBLE com regras condicionais para corrigir inversões de coordenadas. Por fim, informações faltantes foram padronizadas para "NÃO INFORMADO" via COALESCE e NULLIF.

Para a modelagem dimensional, utilizou-se SELECT DISTINCT para extrair os atributos únicos das dimensões e a função ROW_NUMBER() para gerar dinamicamente as chaves primárias artificiais (sk_).

Na construção do modelo central, as regras de negócio de classificação (como categorizar veículos e causas de acidentes) foram aplicadas via CASE WHEN em estruturas de CTEs, permitindo os JOINS sucessivos com os blocos de dimensão. O dbt inferiu automaticamente a árvore de dependências (DAG) através da função {{ ref() }}.

No encerramento, foram aplicados testes automatizados de qualidade de dados (garantindo unicidade, campos não-nulos e integridade referencial) antes de descartar as colunas descritivas da tabela fato, restando apenas as chaves artificiais e as métricas.

3. Justificativa da Escolha da Base de Dados:

Acidentes de trânsito são um problema crítico no meio público e de mobilidade urbana. Sendo assim, o Data Warehouse possui valor real prático, possibilitando análises de Business Intelligence (BI) para identificar pontos cegos na cidade, avaliar o envolvimento de diferentes tipos de veículos e, dessa forma, ajudar na formulação de políticas de prevenção.

A base de dados de Acidentes de Trânsito de Recife (2014 a 2016) possui alta fragmentação e “sujeira” nos dados. Isso se dá pois a formatação e nomes (até quantidades) das colunas variam bastante de acordo com o arquivo, além de existirem arquivos mensais e anuais com formatos diferentes

(GeoJSON e CSV). A escolha desse banco de dados reflete a realidade caótica de lidar com dados diversos e como podem diferir ao longo do tempo. Cada registro é um evento isolado que contém métricas quantitativas claras, o que permitiu uma modelagem dimensional direta em uma Tabela Fato central.

É importante ressaltar que nos estágios iniciais do projeto, exploramos os dados presentes em cada arquivo e suspeitamos a existência de forte redundância nos arquivos de 2014. Após análise mais aprofundada documentada no github em “[pipelns/data/2014/Análise de conflito.md](#)”, decidimos não utilizar os arquivos GeoJSON de 2014 pois as únicas ocorrências que não estavam contempladas no CSV são anomalias nos acidentes de dezembro de 2014 cujas datas indicam terem acontecido em janeiro, não dezembro.

4. Descrição dos Processos de Transformação Aplicados:

ETL (Extração, Transformação e Carga):

- Padronização de datas:

As 8 fontes apresentavam múltiplos formatos de data (incluindo %Y-%m-%d, %d/%m/%Y, %d/%m/%y e %-m/%-d/%Y). Usamos a função ‘to_datetime’ da biblioteca pandas para transformar o elemento ‘data’ de texto para o tipo datetime, tratando as variações de formato entre os anos. Fizemos uma pequena função para limitar os registros com datas nulas ou inválidas (fora do período de 2014-2016) e para gerar uma coluna adicional relacionada ao dia da semana do ocorrido.

- Padronização de texto:

Utilizamos as funções .upper() e .strip() do Python para converter os campos textuais para letras maiúsculas e eliminar espaços em branco extras no início e no final das strings. Além disso, os elementos textuais ausentes na base foram normalizados como 'NÃO INFORMADO'.

- Regras de negócio (classificações):

Criamos as funções:

- **classificar_ocorrencia:** Categoriza os tipos de ocorrências a partir de padrões textuais e palavras-chave que remetem a cada uma das classificações.
- **classificar_causa:** Agrupa e cruza os dados de ocorrência, tipo e descrição, gerando uma visão mais detalhada sobre a causa do acidente.
- **classificar_veiculo:** Identifica e padroniza o tipo de veículo envolvido, utilizando buscas por substrings no texto
- **categorizar_veiculo:** Agrupa as classificações obtidas na função ‘classificar_veiculo’ em macrocategorias (como Motorizado de duas rodas, Não motorizado, etc.), facilitando a análise dos dados.

- **Geração de chaves artificiais:**

Foram criadas surrogate keys(sks) para atuar como chave primária de cada dimensão. Essas chaves foram geradas de forma automática e autoincremental, garantindo a integridade e independência dos dados conforme pede o modelo estrela.

- **Padronização de dados faltantes:**

Nos registros em que a quantidade de vítimas do acidente não estava informada, definimos o valor padrão como 1, assumindo a ocorrência de pelo menos uma vítima por evento, visto que os dados são referentes a acidentes com vítimas.

- **Unificação e modelagem estrela:**

Estruturamos os dados tratados no modelo estrela, distribuindo-os em 5 tabelas dimensão. Essas dimensões foram vinculadas a uma tabela fato do acidente por meio de suas surrogate keys que servem como chaves estrangeiras(compondo a primary key do fato). Depois disso, para exportar os dados para nosso .db, transformamos novamente essas tabelas em csvs.

ELT (Extração, Carga e Transformação):

- **Padronização de datas:**

As 8 fontes apresentavam múltiplos formatos de data (incluindo %Y-%m-%d, %d/%m/%Y, %d/%m/%y e %-m/%-d/%Y). A função `try\_strptime` foi aplicada combinada à `COALESCE` para processar cada formato, convertendo tudo para o tipo nativo `DATE`. Registros com data nula ou fora do intervalo 2014–2016 foram descartados com filtros `WHERE`.

- **Padronização de texto:**

As funções `upper` e `trim` foram aplicadas sobre campos textuais para remover espaços nas extremidades e garantir caixa alta. Espaços múltiplos internos no campo de endereço foram higienizados e convertidos em espaços simples através da expressão regular `regexp_replace(endereco, '\\s+', ' ', 'g')`. Adicionalmente, strings totalmente vazias foram normalizadas para o literal "NÃO INFORMADO" via `COALESCE(NULLIF(campo, ''), 'NÃO INFORMADO')`.

- **Tratamento de coordenadas geográficas:**

As colunas de latitude e longitude que utilizavam vírgula como separador decimal foram corrigidas via `replace(campo, ',', '.')` antes da conversão para `DOUBLE`. Para contornar os registros que continham os valores de eixos invertidos entre si, aplicou-se uma regra de corte condicional baseada na posição geográfica real de Recife (onde a latitude se posiciona próxima a -8°): caso o valor de latitude fosse estruturalmente maior que -8, identificava-se a troca de eixos e o script realizava a inversão dinâmica dos campos.

- **Regras de negócio (classificações):**

Utilizando `CASE WHEN` com `LIKE`, foram criadas classificações derivadas:

- `tipo\_veiculo`: agrupa os valores brutos em categorias como Motocicleta, Automóvel, Ciclista, Pedestre, entre outras.

- ``tipo_ocorrencia``: normaliza o campo de ocorrência bruta em categorias como Colisão, Atropelamento, Perda de controle, entre outras.

- ``causa_acidente``: cruzando ``ocorrencia_raw`` e ``tipo_raw``, classifica o acidente em categorias como Atropelamento de pedestre, Colisão entre veículos, Capotamento/Tombamento, entre outras.

- **Geração de chaves artificiais:**

As chaves primárias (``sk_``) de cada dimensão foram geradas dinamicamente com ``ROW_NUMBER() OVER (ORDER BY ...)``, sem depender de identificadores das fontes originais.

- **Unificação e modelagem estrela:**

A staging ``stg_acidentes`` consolidou as 8 fontes brutas via ``UNION ALL``. A partir dela, cada dimensão isolou seus atributos correspondentes via ``SELECT DISTINCT``, aplicando a função analítica `ROW_NUMBER() OVER (ORDER BY ...)` para a criação das Surrogate Keys (`sk_``). A tabela ``fato_acidente`` foi estruturada através de JOINS sucessivos às dimensões, descartando as colunas descritivas textuais e retraindo apenas as chaves artificiais, a métrica nativa `quantidade_vitimas` e a métrica calculada de volume `quantidade_veiculos` (adicionada estaticamente como 1 por registro).

- **Qualidade dos dados:**

O dbt executou testes automáticos (``unique``, ``not_null``, ``relationships``) definidos no arquivo ``_marts.yml``, validando a integridade referencial de todas as chaves estrangeiras da tabela fato.

5. Comparativo entre ETL e ELT:

Abordagem ETL (Implementação via Python/Pandas e SQLite)

Nesta abordagem, o motor de processamento foi o próprio ambiente de execução do Python, utilizando a memória RAM para manipular os dados antes de enviá-los ao banco. Os dados brutos foram extraídos e lidos em Data Frames do Pandas. Toda a transformação (conversão de datas, higienização textual, tratamento de nulos e aplicação de regras de negócio via funções `.apply()`) ocorreu em memória. Por fim, as tabelas Fato e Dimensões consolidadas foram carregadas (*Load*) em um banco de dados SQLite.

Entre as vantagens percebidas, estão:

- **Flexibilidade programática:** O Python oferece enorme facilidade para manipular strings complexas e aplicar lógicas condicionais linha a linha (estruturas `if/else` nativas).
- **Depuração interativa:** É muito fácil visualizar o estado dos dados ao longo da transformação utilizando comandos como `display()` ou `head()`.

Em contrapartida, também foi possível encontrar desvantagens, como:

- **Gargalo de memória:** O limite de processamento é a memória RAM

da máquina local (ou do ambiente Colab). Para bases na casa dos gigabytes, o Pandas tradicional apresentaria lentidão ou travamentos (*Out of Memory*).

- **Desperdício de I/O:** Os dados são movidos para fora do banco, processados e devolvidos, gerando tráfego desnecessário.

Abordagem ELT (Implementação via DuckDB e SQL):

Neste modelo, o banco de dados assumiu o papel central, atuando não apenas como armazenamento, mas como o motor de processamento. Nele, o Python foi utilizado apenas como orquestrador para carregar (*Load*) os 8 arquivos brutos diretamente para o DuckDB em um *schema* raw (Staging), sem nenhuma alteração (todas as colunas como *string*). Toda a transformação (*Transform*) ocorreu através de *queries* SQL robustas (CTEs), utilizando expressões regulares, conversões dinâmicas de tipo (*try_strptime*) e funções analíticas (*ROW_NUMBER()*) para gerar o esquema estrela.

Ao longo do processo, foi possível observar vantagens e desvantagens. Entre as vantagens, é possível citar:

- **Escalabilidade e desempenho:** O DuckDB, sendo um banco analítico colunar, processa as agregações e cruzamentos (JOINS) de forma muito mais veloz e paralela do que o Pandas.
- **Governança e rastreabilidade:** A linhagem dos dados é clara. A base bruta (raw) e a base modelada (dw) coexistem no mesmo ambiente, permitindo reproprocessar regras de negócio rapidamente sem precisar reler os arquivos originais.

Já em relação as desvantagens percebidas, é válido comentar da:

- **Complexidade sintática para texto:** Operações complexas de limpeza de strings podem gerar comandos SQL longos e aninhados (como o uso múltiplo de CASE WHEN, COALESCE e *regexp_replace*), exigindo domínio avançado da linguagem estruturada.

Para o escopo deste projeto acadêmico (uma base com milhares de registros de acidentes), a abordagem **ETL atendeu muito bem aos requisitos**, pois o volume de dados cabia confortavelmente na memória e permitiu uma construção limpa em Python.

Entretanto, pensando em um cenário real de um portal de transparência municipal ou em um Data Warehouse que receberia cargas diárias e massivas de dados geolocalizados de trânsito ao longo de décadas, a **abordagem ELT se mostrou muito mais adequada**. O pipeline em DuckDB demonstrou ser mais escalável, imune a limitações de RAM para o tamanho da amostra e totalmente alinhado com as práticas atuais do *Modern Data Stack* (permitindo futura integração fácil com ferramentas como o dbt).

6. Apresentação de Três Análises e Insights:

Foram realizadas cinco consultas analíticas sobre o Data Warehouse de

acidentes, utilizando a tabela fato fato_acidente e as dimensões de tempo, local, veículo e ocorrência. O objetivo foi observar os dados por diferentes perspectivas e identificar padrões relevantes sobre os acidentes com vítimas.

A primeira consulta analisou os acidentes por ano e tipo de veículo, mostrando a quantidade de acidentes, o total de vítimas e a média de vítimas por acidente. O resultado indica que, em 2014, os automóveis aparecem como principal categoria, com 754 acidentes. Já em 2015 e 2016, as motocicletas passaram a liderar com ampla diferença, registrando 1504 acidentes em 2015 e 797 em 2016.

	ano	tipo_veiculo	qtd_acidentes	total_vitimas	media_vitimas_por_acidente
0	2014	Automóvel	754	754	1.00
1	2014	Motocicleta	629	629	1.00
2	2014	Pedestre	178	178	1.00
3	2014	Ciclista	150	150	1.00
4	2014	Ciclomotor	23	23	1.00
5	2014	Outros	3	3	1.00
6	2015	Motocicleta	1504	1645	1.09
7	2015	Pedestre	264	276	1.05
8	2015	Automóvel	186	227	1.22
9	2015	Ciclomotor	129	139	1.08
10	2015	Ciclista	94	98	1.04
11	2015	Outros	49	67	1.37
12	2016	Motocicleta	797	898	1.13
13	2016	Automóvel	174	237	1.36
14	2016	Pedestre	143	166	1.16
15	2016	Ciclista	56	64	1.14
16	2016	Outros	30	51	1.70
17	2016	Ciclomotor	28	29	1.04

[Figura 02 - Resultado gerado pela primeira consulta]

A segunda consulta agrupou os acidentes por dia da semana, indicando se o dia era ou não fim de semana. O maior volume ocorreu na sexta-feira, com 859 acidentes, seguida por segunda-feira e terça-feira. Os fins de semana não apresentaram os maiores valores absolutos, sugerindo que os acidentes estão bastante associados à rotina de deslocamento urbano.

	dia_semana	fim_de_semana	qtd_acidentes	total_vitimas	media_vitimas_por_acidente
0	Sexta-feira	Não	859	912	1.06
1	Segunda-feira	Não	788	867	1.10
2	Terça-feira	Não	758	818	1.08
3	Quinta-feira	Não	731	796	1.09
4	Sábado	Sim	730	786	1.08
5	Quarta-feira	Não	704	770	1.09
6	Domingo	Sim	621	685	1.10

[Figura 03 - Resultado gerado pela segunda consulta]

A terceira consulta identificou os bairros com maior número de acidentes. Boa Viagem foi o bairro com maior volume, com 238 acidentes e 271 vítimas, seguido por Imbiribeira e Santo Amaro. Também foi possível observar que alguns bairros, como Caxangá, Imbiribeira e Madalena, apresentam médias mais altas de vítimas por acidente.

	bairro	qtd_acidentes	total_vitimas	media_vitimas_por_acidente
0	BOA VIAGEM	238	271	1.14
1	IMBIRIBEIRA	153	191	1.25
2	SANTO AMARO	135	157	1.16
3	BOA VISTA	86	107	1.24
4	CASA AMARELA	85	101	1.19
5	MADALENA	84	105	1.25
6	AFOGADOS	84	87	1.04
7	CORDEIRO	75	87	1.16
8	CAMPO GRANDE	72	86	1.19
9	IBURA	71	82	1.15
10	IPSEP	60	66	1.10
11	TORRE	59	66	1.12
12	SÃO JOSÉ	59	67	1.14
13	CAXANGÁ	58	76	1.31
14	DERBY	56	63	1.13

[Figura 04 - Resultado gerado pela terceira consulta]

A quarta consulta analisou os acidentes por tipo de ocorrência e ano. Em 2014, a maior parte dos registros aparece como “Outros”, provavelmente pela menor padronização da base naquele ano. Em 2015 e 2016, as colisões passaram a ser a categoria predominante, com 1182 e 1057 acidentes, respectivamente.

	ano	classificacao_ocorrencia	qtd_acidentes	total_vitimas
0	2014	Outros	1735	1735
1	2014	Colisão	2	2
2	2015	Colisão	1182	1394
3	2015	Outros	876	877
4	2015	Atropelamento	153	162
5	2015	Perda de controle	10	13
6	2015	Atropelamento de animal	5	6
7	2016	Colisão	1057	1239
8	2016	Atropelamento	148	174
9	2016	Outros	13	13
10	2016	Perda de controle	9	18
11	2016	Atropelamento de animal	1	1

[Figura 05 - Resultado gerado pela quarta consulta]

Por fim, a quinta consulta cruzou o tipo de ocorrência e o tipo de veículo. O principal padrão encontrado foi a combinação entre colisão e motocicleta, com 1716 acidentes e 1949 vítimas. Também se destacam atropelamentos envolvendo pedestres e colisões envolvendo automóveis.

	classificacao_ocorrencia	tipo_veiculo	qtd_acidentes	total_vitimas	media_vitimas_por_acidente
0	Colisão	Motocicleta	1716	1949	1.14
1	Outros	Motocicleta	1195	1196	1.00
2	Outros	Automóvel	825	825	1.00
3	Outros	Pedestre	284	284	1.00
4	Atropelamento	Pedestre	280	309	1.10
5	Colisão	Automóvel	277	372	1.34
6	Outros	Ciclista	190	190	1.00
7	Colisão	Ciclista	103	115	1.12
8	Outros	Ciclomotor	95	95	1.00
9	Colisão	Ciclomotor	85	96	1.13
10	Colisão	Outros	41	79	1.93
11	Outros	Outros	35	35	1.00
12	Colisão	Pedestre	19	24	1.26
13	Atropelamento	Motocicleta	10	16	1.60
14	Perda de controle	Automóvel	9	18	2.00
15	Atropelamento	Ciclista	7	7	1.00
16	Perda de controle	Motocicleta	7	9	1.29
17	Atropelamento	Automóvel	3	3	1.00
18	Atropelamento de animal	Outros	3	3	1.00
19	Atropelamento de animal	Motocicleta	2	2	1.00
20	Perda de controle	Outros	2	3	1.50
21	Atropelamento	Outros	1	1	1.00
22	Atropelamento de animal	Pedestre	1	2	2.00
23	Perda de controle	Pedestre	1	1	1.00

[Figura 06 - Resultado gerado pela quinta consulta]

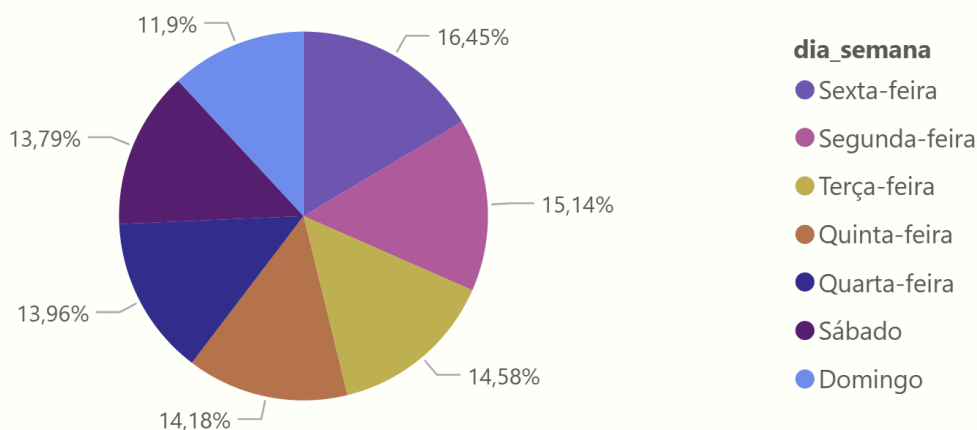
]

[Figura 07 - Gráfico da análise entre tipo de veículo e quantidade de acidentes]

[Figura 08 - Gráfico da análise entre volume de acidentes e média de vítimas por bairro]

O terceiro insight é que os acidentes não estão concentrados apenas nos fins de semana. A sexta-feira foi o dia com maior número de registros, e os demais dias úteis também apresentaram valores elevados. Isso indica que **os acidentes com vítimas parecem estar associados à dinâmica cotidiana da cidade**, como deslocamentos para trabalho, estudo e serviços, e não apenas a eventos típicos de fim de semana. Esse resultado pode ser útil para orientar ações preventivas em dias de maior circulação urbana.

Acidentes X Dia da Semana



[Figura 09 - Gráfico da análise entre acidentes e dia da semana]

7. Reflexão sobre o Aprendizado

O desenvolvimento deste Projeto de Integração proporcionou uma experiência prática sobre a complexidade e a importância da engenharia de dados no mundo real. A manipulação de uma base de dados real, caracterizada por alta fragmentação, inconsistências estruturais e ruídos informacionais, exigiu a consolidação dos conceitos teóricos vistos em sala para a aplicação em um cenário mais próximo da realidade.

Entre os desafios superados, é possível citar o maior obstáculo inicial como a alta heterogeneidade das fontes. Foi preciso processar arquivos em formatos distintos (.csv e .geojson), com separadores diferentes, colunas ausentes e nomenclaturas inconsistentes. Padronizar formatos de data divergentes, tratar valores nulos e corrigir eixos invertidos de latitude e longitude exigiu a aplicação rigorosa de expressões regulares, lógica condicional e funções de limpeza (como COALESCE e try_strptime). Além disso, houve uma mudança de paradigma na modelagem, afinal, sair de uma modelagem relacional transacional tradicional (processos da AV1 até AV4) e construir um Data Warehouse em Esquema Estrela exigiu uma nova forma de pensar. O desafio foi compreender na prática a função central da Tabela Fato e o papel das chaves substitutas (Surrogate Keys) nas Tabelas Dimensão para garantir a independência estrutural da base final. Por fim, orquestrar o pipeline através de duas abordagens distintas exigiu dedicação. Enquanto o processo ETL demandou forte domínio de manipulação de DataFrames em memória com Python e Pandas, a abordagem ELT nos desafiou a

dominar o ecossistema do dbt aliado ao DuckDB para garantir a linhagem e testagem da qualidade dos dados.

Entre as lições aprendidas, é válido comentar como ficou claro que dados sem padronização perdem seu valor analítico. A criação de mapeamentos (dicionários) logo no início do projeto e a definição de regras claras de negócio (como a categorização de veículos e tipos de acidentes) foram passos cruciais para que a junção (UNION) dos dados não gerasse um modelo inconsistente. Adicionalmente, a execução em Python (Pandas) lado a lado com a execução em SQL moderno (dbt/DuckDB) mostrou que não existe uma "ferramenta ideal", mas sim abordagens mais adequadas para diferentes cenários de escalabilidade e governança.

Portanto, é válido ressaltar que o projeto atingiu seu objetivo principal ao transformar dados brutos e desorganizados em uma base limpa e pronta para a realização de análises e tomada de decisões. Para além do desafio técnico, a integração dos dados permitiu visualizar de forma clara problemas reais do trânsito de Recife, como a grande exposição dos motociclistas a riscos de colisão e a variação da gravidade dos acidentes de acordo com o bairro, validando o propósito prático e analítico do trabalho realizado.